Flash Attention, Slowly

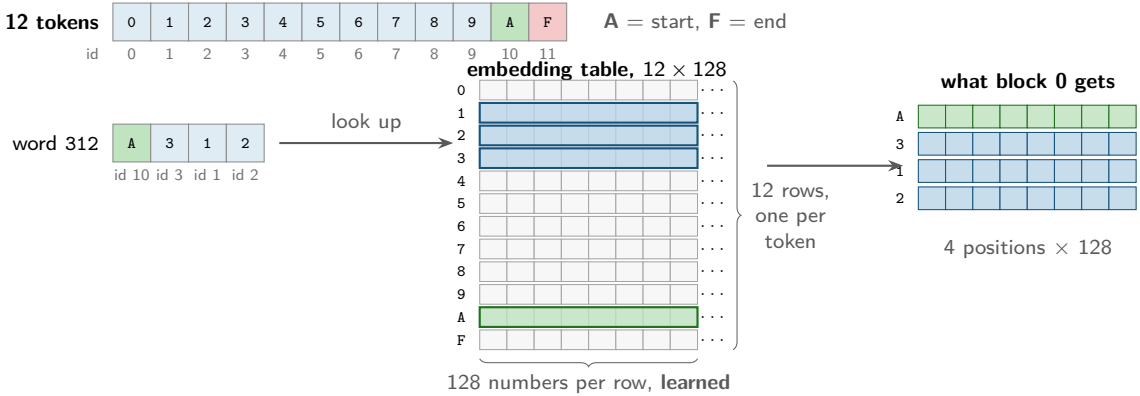
Starting from TinyGPT

SC26 nixnan tutorial

`flash-tutorial`

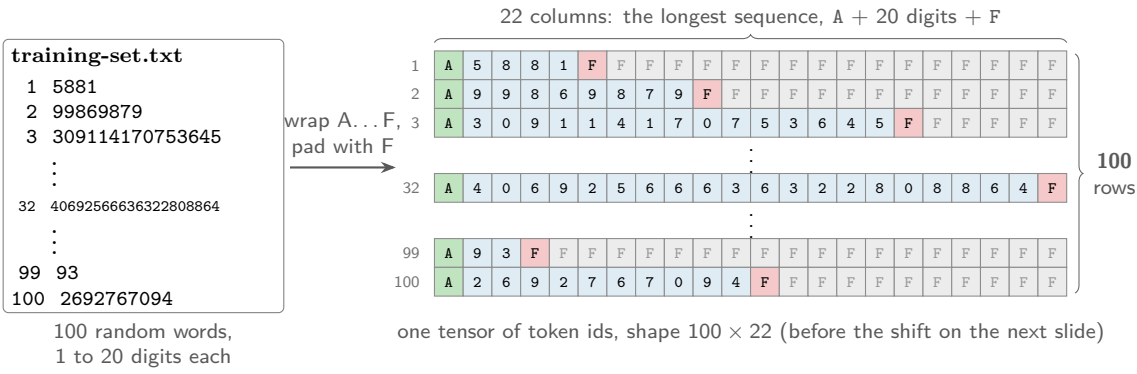
October 2026

A token is a row number, not a number



- The network never sees that 3 means three. To it, 3 is whatever vector sits in row 3; training moves the rows.
- The code multiplies a one-hot matrix by the table (same rows, same FP32 values) instead of calling `nn.Embedding`: the lookup kernel hangs NVBit.

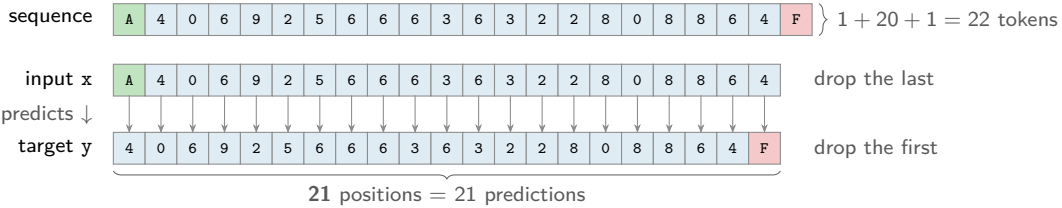
Where the batch size 100 comes from: one row per training word



There is no batch-size setting: `train.py` stacks *every* word of the training set into one batch and takes all 1500 optimizer steps on that same batch (no shuffling, no sampling).

Where 21 comes from: each position predicts the next token

word 32 (20 digits, the longest)

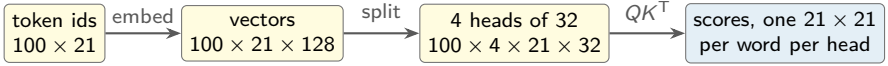


word 1 (5881): padded to the same 21

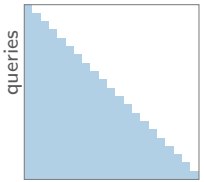


So the training batch is 100×21: 100 words, 21 positions each.

The batch on its way into attention

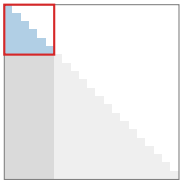


word 32: all 21 positions real



query i sees keys $0..i$ (causal)

word 1 (5881): 6 real, 15 padding



real queries see only real keys:
a pad comes *after* every real token,
so causal masking hides it.
No padding mask is needed.

pad queries (gray) are
computed, then ignored by the loss

Training (`train.py`): $100 \times 4 = 400$ score grids of 21×21 in each of the 4 blocks, every step, on the same batch for all 1500 steps.

Generation (`generate.py`): batch **1**. Prompt `A312` is one call ($1 \times 4 \times 4 \times 32$), then one call per new token, attending over the stored keys: up to 7 positions.